CSA02 – ASSIGNMENT 1

Grama Jyothi Rural Electricity Co-operative
C Programming: Complete Answers with C Code

Name: BAVADARANI S

Register Number: 1912260562

GitHub Repository Link: https://github.com/bavadarini28/ CSA0209-1912260562-BAVADARANI-S-ASSIGNMENT-1.git

Scenario and Design

Grama Jyothi processes monthly electricity readings. The program must calculate consumption correctly, handle a 16-bit meter rollover from 65535 to 0, use a single definition for tariff constants, validate input, and avoid floating-point errors in currency. The example tariff used in this report is: first 100 units free; units 101–200 at Rs 2.25 per unit; units above 200 at Rs 4.50 per unit. Since the question does not specify a fixed charge, this implementation assumes a fixed charge of Rs 30.00. Change the single constant if the co-operative specifies another amount.

Data types: uint16_t for meter readings, uint32_t for consumption, and int64_t for money in paise. Using integer paise means Rs 2.25 is stored as 225 paise and Rs 4.50 as 450 paise.

TASK 1 – Header Files and Conditional Compilation (3 Marks)

Create two files in Task01: billing.h and billing.c. The header holds constants and prototypes.

File: Task01/billing.h

```
#ifndef BILLING_H
#define BILLING_H

#include <stdint.h>

#define FREE_UNITS 100u
#define SECOND_SLAB_LIMIT 200u
#define SECOND_SLAB_PAISE 225u
#define THIRD_SLAB_PAISE 450u
#define FIXED_CHARGE_PAISE 3000LL

uint32_t calculate_units(uint16_t previous, uint16_t current);
int64_t calculate_bill_paise(uint32_t units);
void print_money(int64_t amount_paise);

#endif
```

File: Task01/billing.c

```
#include <stdio.h>
#include "billing.h"

uint32_t calculate_units(uint16_t previous, uint16_t current)
{
    /* Unsigned 16-bit subtraction handles meter rollover modulo 65536. */
    return (uint16_t)(current - previous);
}

int64_t calculate_bill_paise(uint32_t units)
{
    int64_t energy = 0;

    if (units > FREE_UNITS) {
        uint32_t second = units < SECOND_SLAB_LIMIT
                        ? units - FREE_UNITS
                        : SECOND_SLAB_LIMIT - FREE_UNITS;
        energy += (int64_t)second * SECOND_SLAB_PAISE;
    }
    if (units > SECOND_SLAB_LIMIT) {
        energy += (int64_t)(units - SECOND_SLAB_LIMIT) * THIRD_SLAB_PAISE;
    }

#ifdef DEBUG
    printf("[DEBUG] units=%u, energy_paise=%lld\n",
           units, (long long)energy);
#endif
    return energy + FIXED_CHARGE_PAISE;
}

void print_money(int64_t amount_paise)
{
    printf("Rs %lld.%02lld\n",
           (long long)(amount_paise / 100),
           (long long)(amount_paise % 100));
}

int main(void)
{
    unsigned int prev_input, curr_input;
    uint16_t previous, current;
    uint32_t units;
    int64_t bill;

    printf("Enter previous reading (0-65535): ");
    if (scanf("%u", &prev_input) != 1 || prev_input > 65535u) {
        fprintf(stderr, "Invalid previous reading.\\n");
        return 1;
    }
    printf("Enter current reading (0-65535): ");
    if (scanf("%u", &curr_input) != 1 || curr_input > 65535u) {
        fprintf(stderr, "Invalid current reading.\\n");
        return 1;
    }

    previous = (uint16_t)prev_input;
    current = (uint16_t)curr_input;
    units = calculate_units(previous, current);
    bill = calculate_bill_paise(units);

    printf("Units consumed: %u\\n", units);
    printf("Bill: ");
    print_money(bill);
    return 0;
}
```

Explanation

A header file (.h) stores declarations shared by source files. main() controls input and output; helper functions calculate units, calculate the bill, and print money.

The include guard (#ifndef, #define, #endif) prevents the same header contents from being processed more than once in a translation unit.

`#include "billing.h" searches the local/project include path first. #include <stdio.h> is used for standard/system headers.`